CS 486/686
Sequence Models, Attention, and Transformers

Yuntian Deng

Lecture 19

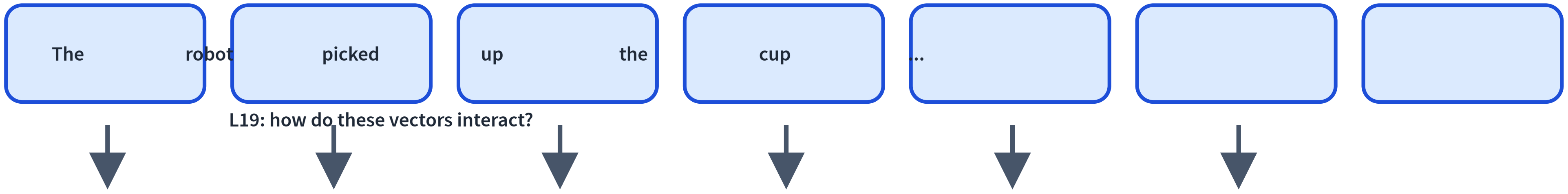
How embeddings talk to each other

Learning goals

- Model a sentence as a **sequence of embeddings**.
- Explain **RNNs** and why they struggle to scale.
- Explain **attention** as token-to-token retrieval.
- Assemble a **transformer block** and understand **causal masking**.

From embeddings to sequences

L18: each item can become a learned vector.



A sentence is not a bag of words

dog bites man

man bites dog

Same words. Different order. Different meaning.

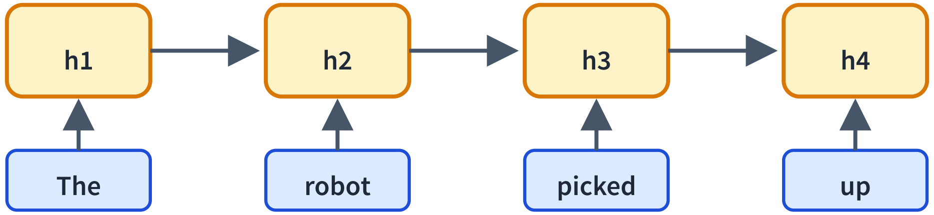
Running example

The robot picked up the cup because **it** was empty.

What does **it** refer to?

The answer depends on context, not the token alone.

The first idea: recurrent state



Read one token at a time.

$$h_t = f(h_{t-1}, x_t)$$

The hidden state summarizes the past.

RNN inference: update and pass forward

The	robot	picked	up
may encode: determiner	may encode: entity	may encode: action	may encode: phrase

The hidden state is one dense vector, not a literal list;
these labels are intuition for what may accumulate.

Why RNNs are limited

Sequential

Step t waits for $t - 1$.

Bottleneck

Everything squeezes into one state.

Long paths

Distant tokens interact through many steps.

Good concept, hard scaling story.

Modern recurrence revisits the idea

Structured state updates can be faster and stabler:

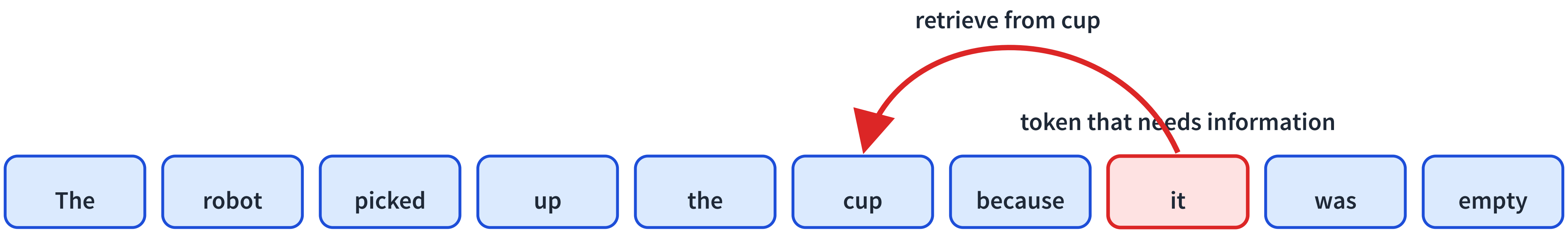
$$\mathrm{state}_t = A\,\mathrm{state}_{t-1} + Bx_t$$

> A is a learned state update; B maps the new input into state.

Modern state-space models keep recurrence, but redesign it for parallel hardware.

Different idea: let tokens look directly

When processing **it**, which earlier word contains useful information?

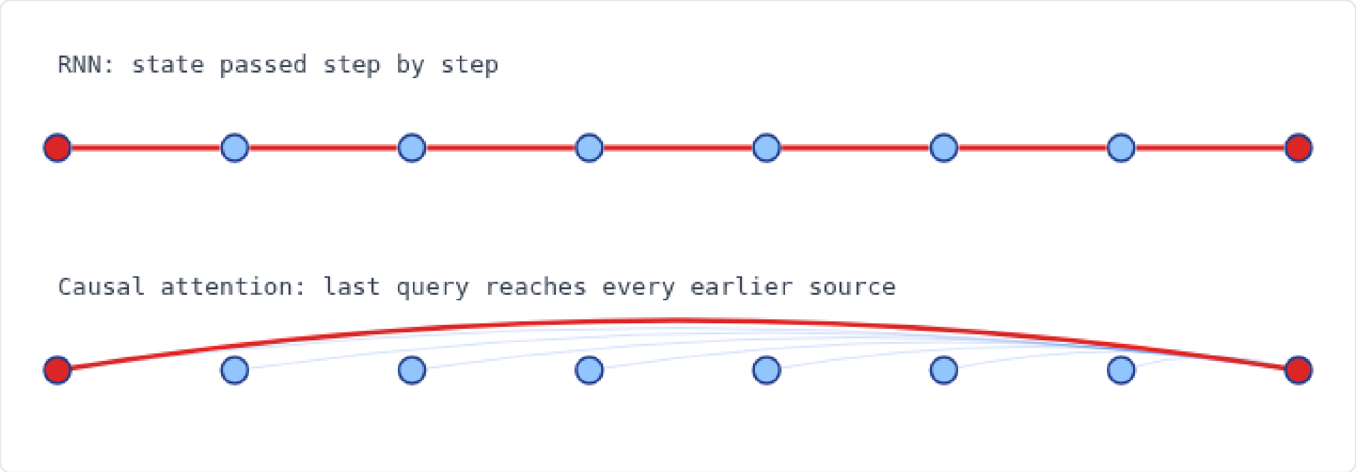


A decoder can look only at itself and earlier source tokens.

Reaching across the sequence LIVE

For an earlier source and a later query, the RNN path grows; attention stays one hop.

sequence length n **8**



`RNN: first ↔ last token is` **`7 steps`** `apart (must go one at a time)`

`Causal attention: last query reaches any earlier source in` **`1 hop`**`; sequence compute remains` **`O(n²)`**

Attention is content-based lookup

1 Match

Compare what *it* needs with each earlier source.

→

2 Retrieve

Bring back a weighted mix of source information.

query
the token doing the attending

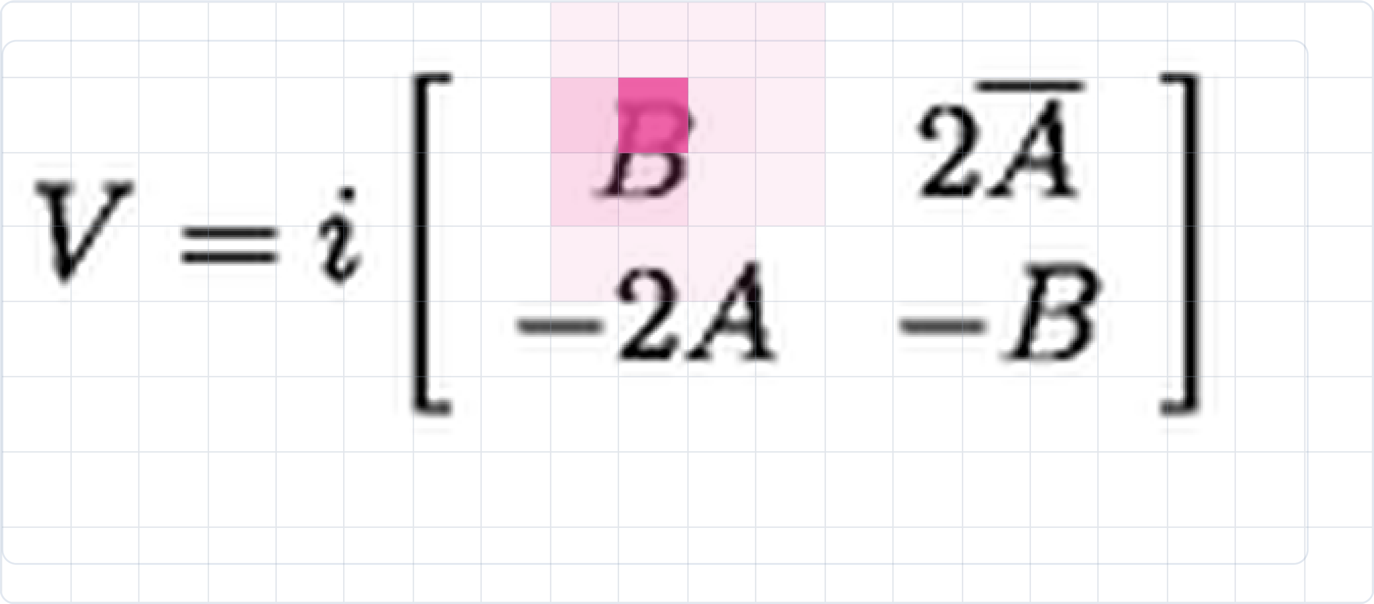
key
each source token’s address

value
that source token’s payload

Keys decide where to look; values decide what comes back.

A visual example: image → LaTeX

Hover a generated LaTeX token. The **pink** map is its spatial attention over the source image.



`V` `=` `i` `[` `B` `2` `\bar` `A` `-` `2` `A` `-` `B` `]`

output token **B** → pink cells show where its visual payload is read

Deng, Kanervisto, Ling & Rush · “Image-to-Markup Generation with Coarse-to-Fine Attention” · ICML 2017 ↗

query

decoder state: what LaTeX symbol should come next?

keys

addresses of spatial image regions

values

visual features read from the attended regions

Cross-attention: Q comes from the decoder; K, V come from the image. **Self-attention next:** Q, K, V all come from the text sequence.

Queries, keys, values are learned projections

At the current layer, token i has hidden state $\mathbf{h}_i$:

need

$\mathbf{q}_i = W_Q \mathbf{h}_i$

query

address

$\mathbf{k}_i = W_K \mathbf{h}_i$

key

payload

$\mathbf{v}_i = W_V \mathbf{h}_i$

value

Same hidden state, three learned views. In deeper layers, $\mathbf{h}_i$ already contains context.

One query scores earlier keys

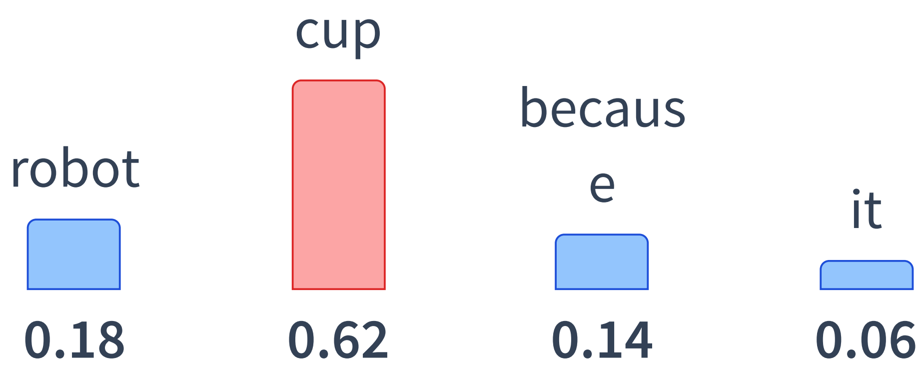
Current token: *it* supplies $\mathbf{q}_{\text{it}}$.

Allowed source j: $j \leq i$; it supplies $\mathbf{k}_j$ now and $\mathbf{v}_j$ later.

$$\text{score}(\text{it}, j) = \mathbf{q}_{\text{it}}^{\top} \mathbf{k}_j$$

source key k_j	**match score**
robot	medium
cup	**high**
because	low
it (self)	low

Softmax turns scores into weights



The allowed source weights are nonnegative and sum to 1.

Use the weights to mix source values

$$\mathbf{z}_{\text{it}} = \sum_{j \le i} \alpha_{\text{it},j} \mathbf{v}_j$$

$0.18\ \mathbf{v}_{\text{robot}}$ | $0.62\ \mathbf{v}_{\text{cup}}$

$0.14\ \mathbf{v}_{\text{because}}$ | $0.06\ \mathbf{v}_{\text{it}}$

→ $\mathbf{z}_{\text{it}}$

new context vector

Because **cup** has the largest weight, its payload contributes most.

Scaled dot-product attention

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

$QK^\top$

all pairwise relevance scores

V

the information being mixed

d_k is the query/key width; dividing by $\sqrt{d_k}$ keeps large dot products from saturating softmax.

Scores → weights LIVE

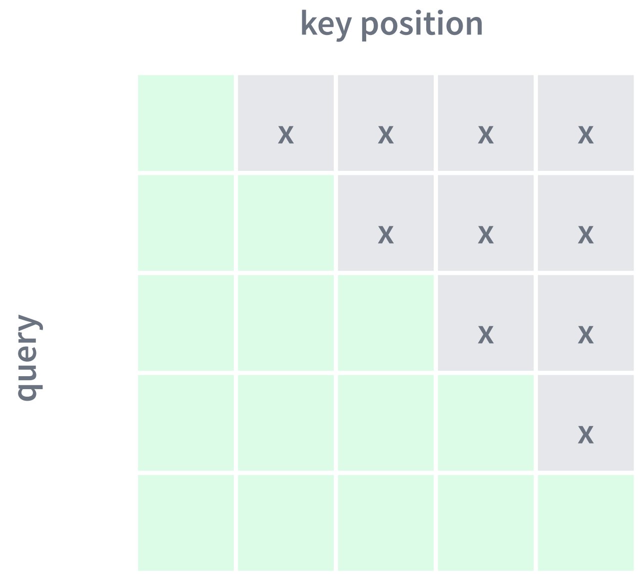
Softmax turns raw scores into attention weights — scaling sharpens or flattens them.

temperature 1.0

dog 1.0 cat 3.0 bird 1.5 fish 0.5

Softmax turns scores into probabilities. Low temperature sharpens; high temperature flattens (the decoding knob in L20/L21).

Causal masking prevents peeking



A decoder predicts the next token from the prefix.

Query position i may use keys/values only from $j \le i$.

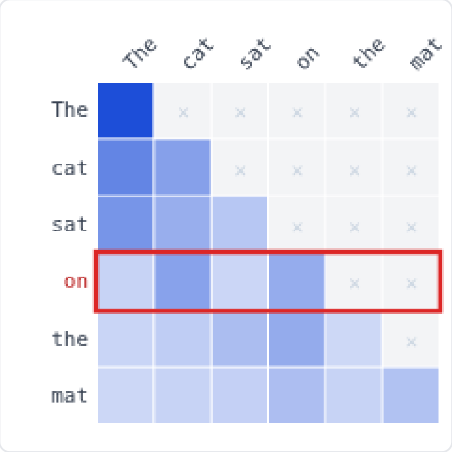
$$\mathrm{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right)V$$

$M_{ij} = 0$ when $j \le i$
$M_{ij} = -\infty$ when $j > i$

Causal masking, interactively LIVE

Toggle the mask and pick a query word — watch the weights renormalize over allowed positions.

causal mask: ON



query: **on** (position 4)

The 0.13

cat 0.40

sat 0.12

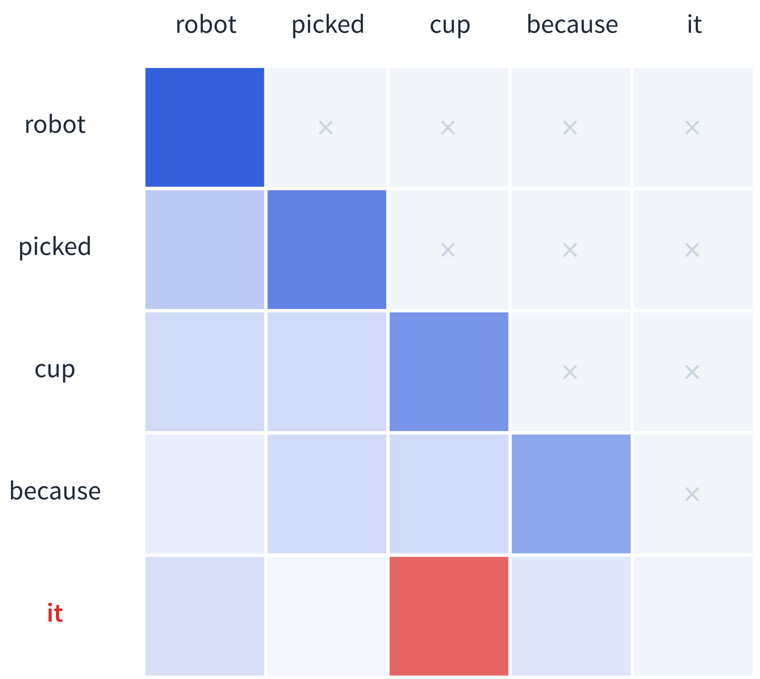
on 0.34

the –

mat –

Future words are blocked, so the weights renormalize over positions ≤ t.

Every row asks where one query looks



Rows: query positions.

Columns: source keys/values.

Toy row: query **it** puts its largest weight on **cup**.

Gray cells are future positions blocked by the causal mask.

Real attention in a decoder LIVE MODEL

Actual Qwen3-0.6B causal attention — pick a head, then click a query word.

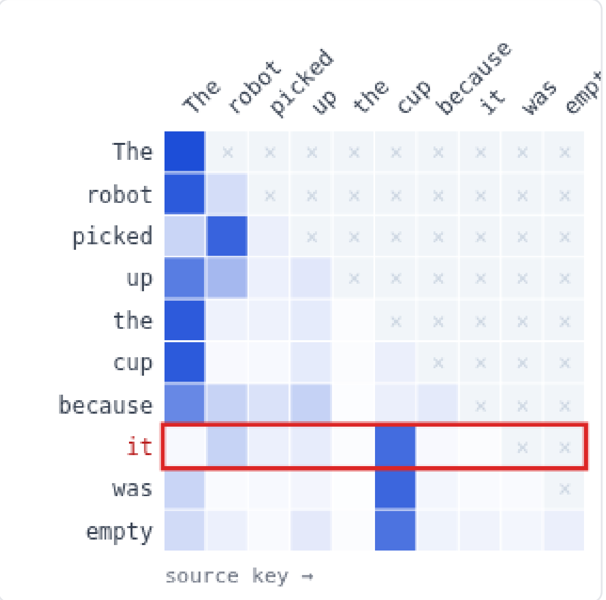
robot / cup

chef / soup

L11 H1: high it → cup

L15 H3: previous token

L1 H12: broad prefix



query: it attends to…

cup	0.77
robot	0.14
up	0.04
picked	0.03
The	0.01
because	0.01

Real Qwen3-0.6B causal attention. Click a query row.

Multi-head attention repeats the lookup

Each head learns different W_Q, W_K, W_V projections.

source link

high *it* → *cup*

local pattern

previous token

broad pattern

spread over prefix

Heads can emphasize different patterns in parallel; the real demo shows measured examples.

Transformers dropped recurrence—where did order go?

RNN

Order is built into sequential updates.

Transformer, layer 1

all token vectors processed in parallel

Content alone does not label first, second, third…

Fix the final query and shuffle the same previous vectors: content-only first-layer attention is unchanged *by word*.

Position changes the attention scores

simple start: $\mathbf{h}_i^0 = \text{token}_i + \text{position}_i$

Now the projected queries and keys depend on both content and slot.

Modern decoders: Qwen/GPT-style models commonly rotate query/key dimensions with RoPE, encoding relative position directly in their match.

Mask: earlier vs future. **Position signal:** richer relative/absolute order.

Can the final token tell who did what? LIVE

Query: q_{because} **Sources:** dog, bites, man **Goal:** different mixed context before predicting next.

add positions: OFF

“dog bites man **because**”

q_{because} attends backward to:

dog	0.27
bites	0.37
man	0.36

“man bites dog **because**”

q_{because} attends backward to:

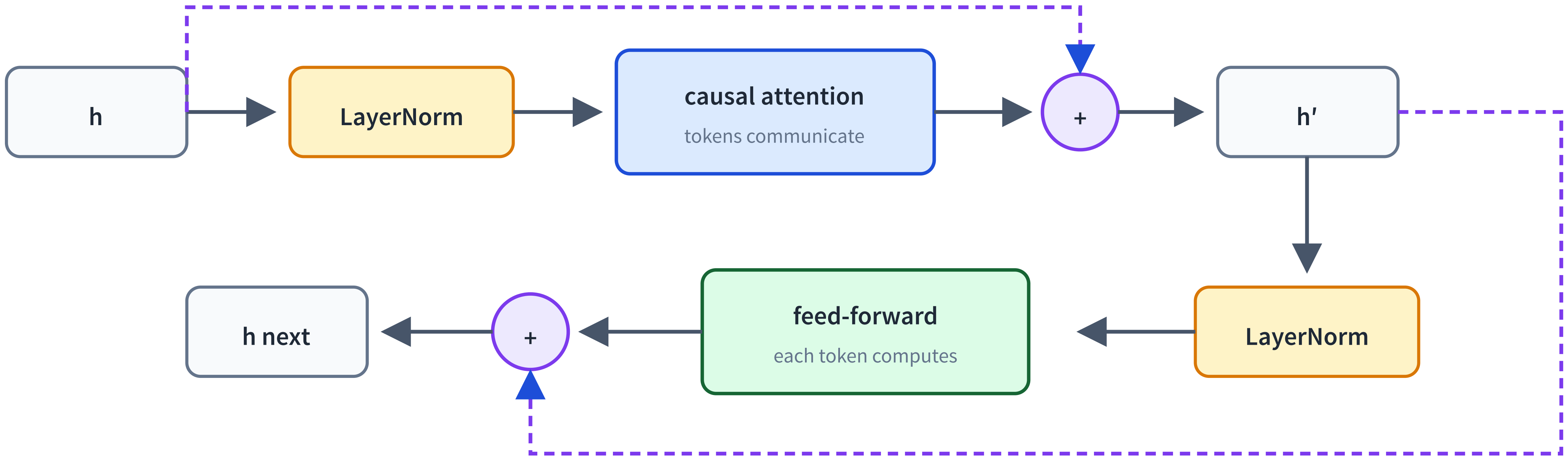
man	0.36
bites	0.37
dog	0.27

$z_1 = 0.27v_{dog} + 0.37v_{bites} + 0.36v_{man}$ **=** $z_2 = 0.27v_{dog} + 0.37v_{bites} + 0.36v_{man}$

Same weighted sum → same context. Layer 1 cannot tell who bit whom.

Deeper layers can differ anyway: each source hidden state was built from a different causal prefix.

A Transformer block: communicate, then compute



LayerNorm stabilizes each sublayer input; residual paths preserve the previous representation.

Keys and values become contextual

input
token + position

→

after block 1
token + retrieved prefix

→

deeper blocks
richer contextual hidden state

Shuffle previous words and their causal histories change—so deeper-layer $\mathbf{k}_j, \mathbf{v}_j$ can change too.

No explicit position encoding does not mean "no position." Causal decoders can infer position from the mask and prefix structure: Haviv et al. (2022); Kazemnejad et al. (2023).

Two common attention masks

Bidirectional encoder

Every token may use both left and right context. BERT-style.

Causal decoder

Position i uses only $j \leq i$. Qwen/GPT-style.

Today's main path is the decoder: its mask enables next-token training and generation.

Why transformers took over

Parallel

All positions at once during training.

Long-range

Any allowed source is one attention hop away.

Scalable

Bigger models and data kept improving.

Now we have the architecture

Next we need the training task:

predict the next token at massive scale

L20: pretraining language models from first principles.

Recap

✓ RNNs summarize the past with a **hidden state**.

✓ Attention retrieves information directly between **tokens**.

✓ Queries, keys, and values implement learned retrieval.

✓ Transformer blocks stack **attention** and **feed-forward** computation.

✓ Causal masks enable autoregressive training and generation.